

Describing Binary Data using Dependent Types

Daniel Hally
2660097h

Abstract

Dependent types allow for the types assigned to data to depend on the actual values of previous data. This can be used to enforce richer compile-time validation of programs than traditional type systems.

Domain-specific languages (DSLs) provide a focused, specialised feature set to aid developers in writing code for a specific task. Embedded DSLs (EDSLs) are implemented inside of an existing programming language, sharing its type system and features.

Data description languages (DDLs) are a type of DSL aimed at describing the structure of data in a rigorous, declarative manner - enabling the generation of parsers, and sometimes serialisers. The data description calculus, DDC^α , is a formalisation of this family of languages with well-defined, robust parsing semantics. It features dependent types, allowing for the values of earlier data to influence the typing of later data - though the host language it generates parsers for is not dependently typed, meaning a lot of this information lost in translation.

This project created an EDSL based on DDC^α in Idris 2, a mature dependently-typed programming language. By using a dependently typed host language, the EDSL is able to preserve the richer type information from DDC^α 's type system in its host language representations of the types. Additionally, using an embedded DSL rather than a standalone one removes the need for external tooling to generate code during compilation, and allows interactions between the host language and the DSL to be fully type checked. The project also extends DDC^α 's semantics with serialisation and implements this in the EDSL.

Compared to previous DDLs embedded in dependently-typed languages, a wider range of parsing behaviours can be expressed and more detailed error reporting is provided.

Using the EDSL, a description of PNG files is defined and an embedding of Fowler [8]'s PacketLang is created.

1 Introduction

While data often comes in standardised formats such as JSON and WAV, programmers still often need to interact with ad-hoc data (i.e. non-standardised formats) such as packets from application-specific network protocols or custom binary file formats (for example, the save file of a video game). While standard formats tend to be thoroughly documented and have well-tested libraries available in a range of languages, ad-hoc formats can be poorly documented and typically have a limited number of public libraries available, if any. This means that programmers are frequently re-inventing the wheel with code for parsing these formats, which is a waste of time and also increases the risk of flaws in the parsers. Given that parsers are often used with external input data, parser bugs can be a serious security risk.

Data description languages (DDLs) are an attempt at addressing these issues, where high-level descriptions of data's structure are written in a custom DSL (a domain-specific language - one designed for a specific task rather than general-purpose computation) and then used to generate parsers. Their conciseness can

allow for them to serve as living documentation of the format at the same time as defining the parser, preventing documentation from becoming outdated or missing specific details implemented in the parser [6]. Generating the parsers from these descriptions reduces the room for bugs, since the need to manually manipulate the raw input data is removed. Some DDLs are also portable, supporting parser generation for multiple different programming languages.

DDLs can be useful for standardised formats too: defining parsing declaratively can make it easier to check against the original specification than a manually written parser, and generating parser code is again useful to reduce the risk of bugs in manipulating the data.

Fisher et al. [7] created a data description calculus, DDC^α , to formalise this family of languages, facilitating comparison between them and helping measure their expressivity. DDC^α defines a type system where the types describe both how to parse the data and how to represent the parsed data. It is provably robust, making it useful for formally verifying the safety of data description languages, as well as creating robust parsers using the calculus itself. The DDC^α type system is dependent, meaning that the typing of data can depend on the values of previous data.

Many DDLs are standalone DSLs, which exist outside of any general-purpose programming language (GPL) and are either translated into or interpreted by a GPL. Embedded DSLs (EDSLs), on the other hand, exist inside of a host GPL, sharing its type system and parser.

Idris 2 [3] is a functional, dependently-typed, general-purpose programming language. Dependently typed languages are particularly suited to hosting EDSLs, allowing stronger compile-time validation of the code interacting with the DSL (for example, the precise result type of parsing data using a data description EDSL can be known at compile-time).

This project brings together 2 lines of work - generic data description and richer EDSLs in dependently-typed languages - by creating an EDSL based on DDC^α in Idris 2 for robust parsing with strong compile-time safety guarantees.

The following shows an example of the EDSL being used to describing a simple binary format, consisting of a 32-bit unsigned integer length and an ASCII string of said length:

```
boundedString : DDCType
boundedString = do
  len <- Buint32
  Pstring_FW (cast len)

serialiseExample : Binary
serialiseExample = serialise boundedString (5 ** "12345")

covering parseExample : IO ()
parseExample = do
  let (_, res) = parse boundedString serialiseExample 0
  let Yes _ = res.ec `decEq` OK
  | No _ => putStrLn "Parse failed"
  let (len ** msg) = stripMetadata boundedString res
  putStrLn $ (show len) ++ " length message: " ++ msg.value
```

Executing `parseExample` prints "5 length message: 12345".
The project makes the following contributions:

- An embedded DSL in Idris 2 based on DDC^α , covering all features apart from recursive types
- Type-safe parsing of binary data using dependent types
- Rich host-language types for data described with the DSL, enabled by dependent types
- Detailed error information for failed parses
- An extension of DDC^α 's semantics with serialisation, implemented in the DSL
- Generation of DSL base types for any Idris type implementing the `FiniteBits` interface
- A description of PNG in the DSL
- A mapping of Fowler [8]'s `PacketLang` to DDC^α constructs
- A feature-complete embedding of `PacketLang` into the DSL
- Proof-by-construction that errors are correctly reported, such that checking the outermost error code is sufficient to guarantee an entire parse was successful.

Notably, the project did not focus on performance optimisation, only correctness.

2 Background

2.1 Domain-Specific Languages

A domain-specific language is one designed for a specific use case, aiming to provide greater expressiveness and ease-of-use than a general purpose language (i.e. one designed to support any computation) would for the same application [14]. DSL programs can also be easier to reason about than programs written in GPLs, given the language's more limited, higher-level feature set, which can aid in efforts to verify their correctness [9].

An example of a widespread DSL is SQL. It allows declaratively describing database commands with much less code than manually writing out the logic to perform the operations would require [11]. The following is an example of an SQL query [11]:

```
SELECT ID, sequence, protocol FROM oligo
WHERE protocol = 2 AND direction = 'Forward';
```

The ID, sequence and protocol fields are retrieved from table 'oligo' for all entries which match the described constraints. This declarative style is much simpler to write than GPL code that iterates through the table while manually checking the constraints and building up a result list.

Standalone DSLs exist outside of any general-purpose programming language, using their own custom parser and type system. This gives full freedom in designing the syntax and types of the language. To use these languages, they are either translated into or interpreted by a general-purpose programming language.

SQL is classed as a standalone DSL. It has interpreters in several GPLs with support for a variety of underlying databases, meaning that learning a single DSL can allow for working with a variety of database technologies across multiple programming languages.

Embedded DSLs (EDSLs), on the other hand, are created inside of existing general-purpose programming languages using the language's constructs. These can require less effort to create since an existing parser and type system are used, and they can provide increased safety guarantees to the code calling them since they share a type system. Additionally, making use of host language constructs for features such as functions can allow for more advanced features while adding less complexity to the

underlying DSL implementation. However, the richer possibilities for error reporting with a custom parser are lost, and tying a DSL to a specific programming language does reduce portability - though the language a DSL is embedded in does not necessarily have to be the language the final program is executed in, the host language may just be used to generate code in other languages [1].

Functional programming languages can be particularly well-suited to hosting EDSLs, with features like polymorphic types and higher-order functions aiding in structuring the representation of the language [9].

Feldspar is an example of an EDSL, embedded in Haskell. It is designed for high-level programming of digital signal processing algorithms, with these high-level descriptions being used to translate the algorithms into optimised C code. It aims to reduce development time and its limited feature set allows for greater optimisation compared to a GPL at a similarly high level of abstraction. The programs are built up as a data structure in Haskell, which then guides the translation into C code [1].

The following is an example algorithm in Feldspar [1]:

```
square :: Data Int -> Data Int
square x = x*x

sumSq :: Data Int -> Data Int
sumSq n = (sum . map square) (1..n)
```

The polymorphic `Data a` type represents a Feldspar program that computes a value of type `a`; the language is able to make use of various Haskell features such as function definition and composition.

2.2 Idris 2 and Dependent Types

Idris 2 [3] is one of the few mature programming languages with a full-spectrum dependent type system. Dependent types allow the typing of data to depend on the actual values of previous data. For example, the type of a tuple could be set such that the first item stores an integer length, and the second item stores a string of said length. Additionally, types are first-class values that can be manipulated like any other, allowing for functions to compute types. Full-spectrum dependent types mean there are no restrictions on the values which can appear in types, enabling types such as vectors which store their length in the type. This type system enables stronger compile-time validation than traditional type systems.

Unlike other dependently typed languages such as Agda and Coq, Idris is designed with systems programming in mind. This makes it well-suited for a verified binary parsing EDSL. In terms of features and syntax, it is quite similar to Haskell - the main differences between the two are the type systems, and that Haskell uses lazy evaluation while Idris is eager.

2.2.1 Internal Constraints. As an example of the benefits of dependent types, consider a Haskell list append function:

```
append :: [a] -> [a] -> [a]
append [] ys = ys
append (x:xs) ys = x : (append xs ys)
```

The type signature says nothing about the lengths of the inputs or output, so the compiler would have no issue with the implementation simply returning an empty list for all inputs.

Dependent types enable Idris to define the `Vect n a` type, a fixed-length list with n elements of type `a`. This can be used to give the function a more specific type:

```
append : {m : Nat} -> {n : Nat} ->
  (xs : Vect m a) -> (ys : Vect n a) -> Vect (m + n) a
append [] ys = ys
append (x::xs) ys = x :: (append xs ys)
```

The type now encodes that the output length is the sum of the two input lengths. This constraint is verified at compile time, and will give a compiler error if the implementation does not respect it.

2.2.2 Syntax. Syntactically, the most notable difference between Idris and Haskell is that usage of the `:` and `::` operators have been swapped, with Idris using `:` for type annotations and `::` for list construction. Most other Haskell syntax is available in unchanged in Idris.

As a part of the dependent type system, Idris' function signature syntax is extended to allow naming the parameters, making it possible to refer to their values when defining the types of other parameters or the return value.

Functions' types may also include implicit parameters, given in curly brackets (like `m`, `n` in `append`) or created automatically for any lowercase names used in a type without a prior declaration (for example, `a` in `append` becomes an implicit parameter of type `Type`). Implicit parameters do not need to be explicitly passed at call sites if the compiler is able to infer their values. For example, in `append ['a'] ['b']` the compiler is able to infer that `m`, `n` are 1 and `a` is `Char`. When they can not be automatically determined, the syntax `functionName {paramName = value}` can be used to provide values for them. Inference is usually limited to what can be directly concluded from the parameters provided, but the `auto` keyword can be added to expand this to an expression search over everything in scope.

A name may also be prefixed with the quantity `0` to guarantee that its value exists at compile-time only. This makes it a compiler error to use it in any context that would require it to be read at runtime, and it therefore provably has no runtime overhead.

While algebraic data types can be defined in the same syntax as Haskell, there is also an alternative `data where` syntax for defining types with more of the language's features available:

```
data Example : {n : Nat} -> (v : Vect n Char) -> Type where
  EmptyExample : Example []
  OddLengthExample : {m : Nat} -> (v : Vect (2 * m + 1) Char)
    -> Example v
```

Specifically, this definition is able to create implicit parameters `m` and `n`, and use dependent typing to define the vector lengths based on them. It also demonstrates how certain constructors can be limited to only create instances of the type with specific parametrisations: one creates an `Example` parametrised by an empty vector, and the other which creates one parametrised by any odd-length vector. Notably, it is not possible to construct an instance of `Example` parametrised by an even-length vector other than the empty vector.

Record types also have a different syntax to Haskell:

```
record RecordName (param1 : Type1) ... (paramN : TypeN) where
  constructor ConstructorName
```

```
field1 : TypeName
...
```

They have a single constructor generated with the provided name, which takes a value for each of the declared fields, and automatically generated getter functions for each of these fields (one normal function `fieldName`, and a postfix operator `.fieldName` to allow briefer access to values).

As in Haskell, Idris provides `do`-notation as syntactic sugar for the monadic bind operation `>=`. For example:

```
-- Using do-notation
exampleDo : IO ()
exampleDo = do
  str <- getLine
  putStrLn ("Got message: " ++ show str)

-- Equivalent code using raw bind notation
exampleBind : IO ()
exampleBind = getLine >= \str =>
  putStrLn ("Got message: " ++ show str)
```

However, unlike Haskell, Idris permits usage of `do`-notation with overloads of the `>=` operator that use non-monadic types.

Another key feature of Idris is totality checking: functions can be annotated with the keywords `partial` (may not terminate and may not be defined for all possible inputs), `covering` (may not terminate but is defined for all possible inputs) or `total` (defined and guaranteed to terminate for all possible inputs) to make the compiler verify that their implementations meet the respective requirement, giving an error if not. If no totality keyword is specified, the compiler defaults to checking that they are `covering` - though this project makes use of the `%default total` pragma to default to checking they are `total` instead. The compiler only has a limited ability to prove that a function is `total`, so the `assert_total` function is provided to allow overriding it for a given expression. To ensure that the type checker always terminates, type-level function calls will only be evaluated if a `total` function is being used; calls to `covering` and `partial` functions will instead be treated as constant values and left unevaluated.

2.2.3 Dependent Pairs. Dependent pairs are a variant of tuples where the value of the first element may be used to determine the type of the second. A common example of this is a vector bundled with its length: `(n : Nat ** Vect n a)`. This pattern is often seen in the return type of functions outputting a vector where the length can not be known at the call site, such as when reading the contents from I/O. The type of the first element may be omitted if it can be inferred from its usage in the second type and, like tuples, dependent pairs can be chained together inside one pair of brackets as syntactic sugar for nesting them.

The `Subset` type, with constructor `Element`, is a variant of dependent pairs where the second item has quantity `0`. This is useful for bundling a value with a proof over it.

2.2.4 Proofs as Types. Another important possibility with dependent types is encoding proofs as types. By creating a proof type parametrised by values, and providing a limited set of constructors that make it impossible to construct instances with certain parameter values, instances of a proof type existing can act as proofs over the type's parameters. A key example of this in Idris is the `Equal` data type:

```
data Equal : a -> b -> Type where
  Refl : {0 x : a} -> Equal x x
```

It is parametrised by two values, but its only constructor takes just a single value to create an instance where both parameters are the same. This means an instance of `Equal` existing is proof that its two parameters are equal. For example, `Refl {x=1}` can be used to create an instance of `Equal 1 1`, but no constructor exists that could create an instance of `Equal 1 2`. The operator `=` is provided as an infix shorthand for the `Equal` type. Idris also features the `LT`, `LTE`, `GT` and `GTE` types for proofs of inequalities.

It is important for proofs to be total, as it is trivial to create a non-terminating function that claims to provide an instance of a type. For example:

```
covering fakeProof : 1 = 2
fakeProof = fakeProof
```

2.2.5 Contradictions as Types. Dependent types are also able to encode proofs that a statement is false. The `Void` type has no constructors, making it impossible to create an instance of it. Therefore, a total function from a type to `Void` acts as a proof that no value of the other type may exist either. For example, the ability for `total oneNotEqualTwo : (1 = 2) -> Void` to compile proves that `1 = 2` is false, since any instance of `1 = 2` would make it possible to construct an instance of `Void`. Idris provides the function `Not ty` as a shorthand for `ty -> Void`.

Idris also allows skipping implementations of provably impossible to reach code. If the compiler knows an input is impossible to construct when pattern matching, the `impossible` keyword can be used to skip implementing the case. Similarly, the `void` function can act as a value of any type when provided with an instance of the `Void` - this acts as proof that reaching the current code would cause a contradiction, so no actual implementation to produce a value is needed.

2.2.6 External Constraints and Data Constraints. Proof types can then be used by functions' types to apply external constraints on their parameters beyond just what type they are. For example, the function `briefPrint : (str : String) -> {auto 0 isBrief : length str < 10} -> IO ()` requires that the string provided is shorter than 10 characters. Similarly, they can be used in data structures to place constraints on other fields, such as requiring an integer field is even, or that one string field is shorter than another.

2.2.7 Decidability. Consider a type `ConsecutivePair`, which contains two numbers along with a proof that the second is one more than the first, along with `tryConsecutivePair`, a function that constructs two numbers into a `ConsecutivePair` if possible. The types can be defined as follows:

```
record ConsecutivePair where
  constructor MkConsecutivePair
  a : Nat
  b : Nat
  {auto 0 isConsecutive : a + 1 = b}

tryConsecutivePair : (a : Nat) -> (b : Nat)
-> Maybe ConsecutivePair
```

The expected implementation for `tryConsecutivePair` may be something along the lines of:

```
if a+1 == b then Just (MkConsecutivePair a b) else Nothing
```

However, the `==` operator returning true is not sufficient evidence to create an `Equal` proof. It returns only a boolean, which is much less powerful than the `Equal` type in that it says almost nothing about how `==` should actually behave. For example, the type checker would consider it a valid implementation to return false for all inputs. To solve this, Idris defines the `DecEq` interface, providing the `decEq` function which takes two values and returns proof of either their equality or inequality. Using this, a valid implementation can be created as follows:

```
tryConsecutivePair a b = case a + 1 `decEq` b of
  Yes isConsecutive => Just $ MkConsecutivePair a b {isConsecutive}
  No contra => Nothing
```

`decEq`'s return type is a specialisation for `Equal` of the `Dec` type, which represents general decidable properties. For proof type `prop`, `Dec` defines constructors `Yes` with a proof of `prop` and `No` with a proof `prop` is false. Idris provides implementations of decision functions for other properties too, such as `isLTE` for `Dec (a <= b)`.

2.2.8 Induction recursion. A technique commonly seen with dependent types is induction-recursion - the simultaneous definition of a set and functions on that set [5]. For example, consider the following EDSL for parsing basic data:

```
mutual
data DataFormat : Type where
  NAT : DataFormat
  CHAR : DataFormat
  ARRAY : (length : Nat) -> (elementType : DataFormat)
  -> DataFormat
  THEN : (a : DataFormat) -> (b : ParsedTy a -> DataFormat)
  -> DataFormat

ParsedTy : DataFormat -> Type
ParsedTy NAT = Nat
ParsedTy CHAR = Char
ParsedTy (ARRAY length t) = Vect length t
ParsedTy (a `THEN` b) = (x : ResultTy a ** b x)
```

A data format to parse can be described through an instance of the `DataFormat` type. The basic operations are the `NAT` and `CHAR` constructors, which instruct the parser to read a single natural number or character from the input respectively. The `ARRAY` constructor instructs the parser to parse a set number of consecutive instances of another `DataFormat`, and the `THEN` constructor allows for parsing one `DataFormat` then another from the data following it. The `ParsedTy` function defines what a specific `DataFormat` will output after a successful parse.

Induction-recursion enables the `ParsedTy` function to be used within the definition of `DataFormat` while depending on said definition. This allows the `THEN` constructor to make its second component a function parametrised by the parsed value of the first component, enabling the DSL to refer to values of earlier components in later components. For example, `NAT `THEN` \len => ARRAY len CHAR` parses an integer length followed by a character array of that length.

2.3 Data Description Languages

Data description languages are DSLs used in parsing, designed specifically for specifying the structure (and sometimes semantics) of data. Many data description languages have previously been created, targeting various forms of data with varying degrees of validation. They are typically declarative and type-based, using sequences of types and operators on them to describe both the binary layout and parsed representation of data.

2.3.1 Kaitai Struct. A relatively simple example of a data description language is Kaitai Struct [17], a standalone DSL for portably describing binary data formats and generating their parsers in various programming languages like C++ and Python. Kaitai is mostly focused on parsing, though its implementations for some languages also include serialisation. It uses YAML-like syntax to define types and describe data formats as sequences of these types. Its type system is relatively limited, lacking features such as polymorphic and recursive types. There is slight support for dependency, with a custom expression language that can specify the length of and apply validation constraints to fields based on calculations with the values of previous fields (though this dependency is erased from the type level in generated code). It also features basic error handling, throwing exceptions which include the data item that caused the error.

2.3.2 PADS/C. PADS/C [6] is another standalone DSL for describing the layout and semantics of data, this time only targeting C for code generation. It aims to be concise while still capturing all information that would be included in manually written parsers, so it can be a form of living documentation for the data's format. It also has a focus on rich error information for failed parses, providing a detailed parse descriptor with the parsing status of the entire data structure. The approach is similar to Kaitai: sequencing types and a custom expression language to add constraints and limited data dependency. However, PADS/C has a more powerful type system that aims to be able to describe most formats used in the real world. For example, it features a union system that allows multiple types to be given for the same location, taking the first one that successfully parses. Additionally, it has richer array types that can terminate based on a termination expression or the successful parsing of a terminator type, rather than just being provided a length. Its types can also be parametrised by values (but not by other types), reducing the duplication of code required. PADS/C can also generate tools to convert data in the described formats into XML, facilitating use with languages other than C.

2.3.3 PADS/ML. PADS/ML [12] is a successor to PADS/C, developed alongside *DDC^α* and therefore incorporating some of the ideas into its design. It is another standalone DSL with features similar to PADS/C, but instead generates OCAML code and uses a more functional-style syntax. The type system is extended with polymorphic types (i.e. types can now be parametrised by other types), recursive types and true dependent types, making it a more powerful language and allowing for further deduplication of code. The tool generation system is also extended so that developers can create their own description-independent tools, such as converters to other standard formats like JSON.

2.3.4 PACKETTYPES. PACKETTYPES [13] is a similar standalone DSL, tailored specifically for network packets and using C as a target language. A notable feature of PACKETTYPES is the overlaying system, which allows for packets to be embedded in others

without changing the outer packet's definition. For example, the payload of a UDP packet can be any application-specific protocol, and it would not be ideal to create a union of all of these within the definition of the UDP packet itself - the overlay system allows for other packets to give a specific description of the UDP payload without adding complexity to the original UDP definition. One drawback of PACKETTYPES is that it has minimal error reporting, with packets either parsing successfully or failing entirely.

2.3.5 PacketLang. PacketLang is another DSL focused exclusively on network packets, originally created by Brady [4] and further developed by Fowler [8]. Notably, it was implemented as an EDSL in Idris 1, a very similar approach to this project. The choice of an EDSL means that its parsing and type checking can be handled by Idris, reducing the required work and risk of bugs. Additionally, sharing a type system with the dependently typed host language means that richer compile-time validation can be enforced on code interacting with the DSL. For example, the function to unmarshal a packet from a socket can have its return type depend on the packet description passed to it, ensuring parity between what the code expects as output and what the packet will actually be parsed into. PacketLang again has minimal error reporting, consisting simply of failure or success.

The following shows an example packet description:

```
dns : PacketLang
dns = with PacketLang do
  header <- dnsHeader
  qdcount <- bits 16
  amount <- bits 16
  nscount <- bits 16
  arcount <- bits 16
  questions <- listn (intToNat $ val qdcount) dnsQuestion
  answers <- listn (intToNat $ val amount) dnsRR
  authorities <- listn (intToNat $ val nscount) dnsRR
  listn (intToNat $ val arcount) dnsRR
```

Descriptions written in the EDSL are instances of the PacketLang type. Fields of the packet are described by constructors of the type, such as `listn`, and then these fields are sequenced together to one top-level PacketLang instance using the constructor `»=`. Naming this constructor as the bind operator means that Idris' `do` notation can be used as syntactic sugar for defining these sequenced descriptions. The `mkTy` function is defined over PacketLang to compute the Idris type that will be given for a successful parse of a specific packet. Using induction-recursion between PacketLang and `mkTy`, the `»=` constructor is defined such that its right side is a function of the parsed value of its left side, allowing for the formats of later fields to depend on the values of earlier ones. For example, `qdcount` determines how many questions will be parsed in the DNS example. The result type preserves this dependency by defining `mkTy (t1 »= t2)` as `(x : t1 ** mkTy (t2 x))`.

2.3.6 Generic Packet Descriptions. Van Geest et al. [18] created an EDSL for describing data in Agda, another dependently typed language. These descriptions can be used for both parsing and serialisation of data, with the round-trip property (i.e. parsing the output of a serialisation gives back the original input) formally verified through Agda's theorem proving system. The design of the DSL was grounded in type theory, considering the universe of bit strings that can be described through the operators provided. Similarly to PacketLang, the DSL was implemented through induction-recursion and lacks detailed error reporting.

2.4 The Data Description Calculus

The data description calculus DDC^α [7] is an attempt at formalising the family of data description languages, facilitating comparison between them and analysis of their expressivity. It defines a set of fundamental operations that aim to be able to recreate all functionality seen in data description languages, and formal semantics for them. The semantics are formally proven give type-correct results and be robust, with correct parse descriptors in case of failure. Developers of PADS/C were also involved in the creation of DDC^α and therefore had a major influence on it - in particular, it features a similar parse descriptor system to give rich error information.

As in most data description languages, DDC^α is based on types that describe both how to parse and how to represent pieces of data, and operations on these types. These types support dependency, polymorphism and recursion.

DDC^α is designed as a standalone DSL: it defines its own type system and re-implements language features such as abstraction and application itself. Its semantics are defined in terms of translation into a custom host language (a polymorphic lambda calculus), though the design of the calculus itself is not heavily dependent on this specific host language. One notable influence is that the host language does not support dependent types, and therefore DDC^α is designed to be able to erase all dependency from its host-language representation types.

Since the type system is separate from the host language's, the function $\llbracket \tau \rrbracket_{\text{rep}}$ gives the host language type for the parsed value of DDC^α type τ , and $\llbracket \tau \rrbracket_{\text{PD}}$ gives the host language type of its parse descriptor. Additionally, the $\llbracket \tau \rrbracket_{\text{P}}$ function is defined to generate the parsing function for the type.

As an example of a DDC^α type, take $\tau_1 = \{s : \text{Pstring_FW}(1) \mid s = "a"\}$. τ_1 uses base type Pstring_FW to parse a string of length 1, then uses a constraint to check that its value is equal to the string "a". Host language representation type $\llbracket \tau_1 \rrbracket_{\text{rep}}$ would be $(\text{string} + \text{none}) + (\text{string} + \text{none})$, with the none branches representing the string failing to parse, and the outer sum representing whether the constraint failed (right) or was met (left). Notably, the relationship between the string's value and the outer sum is lost in the host-language type.

2.4.1 Type Definitions. DDC^α comes with two fundamental built-in types: `unit`, which consumes no data and always parses successfully nothing, and `bottom`, which always fails to parse. In addition to these, base types $C(e)$, where e is a host language expression parametrising the type, are left for the user to define - providing their parsing semantics and host-language representation through the functions $\mathcal{B}_{\text{imp}}$ and $\mathcal{B}_{\text{type}}$ respectively.

It then defines various operations that can be performed to create more complex types.

- Dependent sum type $\Sigma x : \tau_1. \tau_2$ parses type τ_1 followed immediately by type τ_2 ; τ_2 may depend on the output from parsing τ_1 . For example, an integer length n , expressed as 3 ASCII characters, followed by a string of length n can be defined as $\Sigma n : \text{Puint16_FW}(3). \text{Pstring_FW}(n)$. As syntactic sugar, the operation may be abbreviated to $\tau_1 * \tau_2$ when τ_2 is independent of the result of τ_1 .
- Sum type $\tau_1 + \tau_2$ attempts to parse the given data as type τ_1 , and if that fails then it instead tries to parse it as τ_2 .
- Intersection type $\tau_1 \& \tau_2$ parses the same piece of data as both types.

- Constrained type $\{x : \tau \mid e\}$ only considers a parse successful if the value satisfies host-language expression e .
- Sequence type $\tau \text{ seq}(\tau_{\text{sep}}, e, \tau_{\text{term}})$ describes a sequence of values of type τ . The elements of this sequence are separated by separator type τ_{sep} (which may be `unit` for consecutive sequences). The sequence terminates when either the termination expression e becomes true (for example, checking if the length has reached a fixed value), terminator type τ_{term} is successfully parsed (for example, a null byte in a C-style string), or the end of the file is reached.
- Value abstraction $\lambda x. \tau$ and application τe allow for parametrising types by host language expressions, enabling creation of reusable complex types - such as a consecutive pair of n -character strings $\lambda n. \text{Pstring_FW}(n) * \text{Pstring_FW}(n)$.
- Type abstraction $\lambda \alpha. \tau$ and application $\tau \tau$ allow for parametrising types by other types, further aiding in code re-use.
- Recursive type $\mu \alpha. \tau$ allows for a type's definition to refer to itself through type variable α .
- Absorbing type $\text{absorb}(\tau)$ parses τ but discards the result from the representation.
- Scanning type $\text{scan}(\tau)$ will repeatedly attempt to parse τ at consecutive offsets until it a successful parse is reached.
- Computation type $\text{compute}(e)$ parses no input data, and instead evaluates directly to host-language expression e .

2.4.2 Agda EDSL. Oury et al. [15] created an EDSL in Agda inspired by a limited subset of DDC^α 's features - specifically base types, sums, dependent sums and `absorb`; all other types went unimplemented. Additionally, they did not implement the parse descriptor system.

Having a dependently typed host language provided some advantages over the original definition: their dependent sum preserved the dependency in its representation type, and they were able to do proof-by-construction within the program itself that the resulting type of the parse function was correct (whereas the original had to do this through an external manual proof). They noted that implementing recursive types could cause difficulties with Agda type-checking to failing to terminate.

3 Implementation

3.1 Project Structure

The DDC directory contains all code defining the DSL itself: `DSL.idr` defines the core data structure and its host-language representation types; `Error.idr` provides the types used in error handling; `Parse.idr` and `Serialisation.idr` define the DSL's parsing and representation semantics respectively; `Binary.idr` contains utilities for working with data as vectors of bits; `BaseTypes.idr` provides some base type implementations for use with the DSL; `Ascii.idr` defines utilities for working with ASCII-only strings [10]; helpers for writing in the DSL are defined in `Syntax.idr` and `SequenceUtils.idr`; `Show.idr` defines debug printing for parse results; `Util.idr` contains various utility functions not tied to the DSL itself. These are exposed for importing through the top-level `DDC.idr` file.

The `PacketLang` directory contains the EDSL embedding of `PacketLang`, exposed for importing through `PacketLang.idr`.

Examples contains the code used in the evaluation of the project for parsing PNG and PacketLang-embedded DNS.

3.2 EDSL Definition

Data types described through the DSL are represented by instances of the `DDCType` data structure. It has constructors corresponding to most components of the original DDC^α type system (as described in Section 2.4.1) with a few omissions: abstraction and application, discussed further in Section 3.3.1, and recursive types, discussed further in Section 3.3.5. An abbreviated version of the type's definition follows:

```
data DDCType : Type where
  UNIT : DDCType
  BOTTOM : DDCType
  BASE_TYPE : ... -> DDCType
  DEPSUM : (t1 : DDCType) -> (t2 : RawTy t1 -> DDCType) -> DDCType
  SUM : (t1 : DDCType) -> (t2 : DDCType) -> DDCType
  INTERSECTION : (t1 : DDCType) -> (t2 : DDCType) -> DDCType
  CONSTRAINED : (t : DDCType) -> (test : RawTy t -> Bool)
    -> DDCType
  SEQUENCE : (tElem : DDCType) -> (tSep : DDCType) ->
    (termCon : (n : Nat ** Vect n (RawTy tElem)) -> Bool) ->
    (tTerm : DDCType) -> ... -> DDCType
  COMPUTE : (expr : a) -> DDCType
  ABSORB : (t : DDCType) -> ... -> DDCType
  SCAN : (t : DDCType) -> DDCType
```

`DDCType` is defined together with the `RawTy : DDCType -> Type` function using induction-recursion. `RawTy` serves a similar purpose to DDC^α 's $\llbracket _ \rrbracket_{\text{rep}}$, computing the host-language type for representing a value of a given `DDCType`. However, `RawTy` is only able to represent a successful parse - all error handling such as the value of base types being summed with none is removed. The induction-recursion allows `DEPSUM`, `CONSTRAINED` and `SEQUENCE` to take functions that operate on the parsed values of their other `DDCType` parameters, with concrete types applied to those functions' input types.

A separate `MkTy : DDCType -> Type` function is defined to provide the host-language type given by parsing a `DDCType`, including all error handling and metadata - effectively a combination of DDC^α 's $\llbracket _ \rrbracket_{\text{rep}}$ and $\llbracket _ \rrbracket_{\text{PD}}$ functions (discussed further in Section 3.3.3). Specifically, it returns a parametrisation of the `Result` type, discussed further in Section 3.5.

Binary data can be parsed based on a `DDCType` using the covering parse : `DDCType -> Binary -> Offset -> (Offset, MkTy t)` function (the equivalent of DDC^α 's $\llbracket _ \rrbracket_{\text{p}}$).

The `stripMetadata : (t : DDCType) -> (res : MkTy t) -> {auto 0 isOk : res.ec = OK} -> RawTy t` function takes a parsing result proven to be successful and simplifies it to its raw representation.

The `serialise : (t : DDCType) -> RawTy t -> Binary` function encodes a raw representation of a `DDCType` to a bit string; using the raw representation here prevents the need to construct arbitrary parse descriptors to be able to serialise data, and guarantees that serialisation is possible for any type-checked input.

Dependent types enable the `parse` and `stripMetadata` functions to be given precise return types matching exactly what the `DDCType` passed will parse into, and similarly enable calculating the exact required input type for serialise.

Alternative syntax is defined for a few of these constructors to aid users: `t1 .+ t2` and `t1 .& t2` can be used to define sum and intersection types respectively, `t # test` can be used

to define constrained types, and `DEPSUM` is aliased as `»=`. The `DEPSUM` alias is the most noteworthy, as it enables descriptions to be defined using `do`-notation (as in `PacketLang`). For example, `Pint `DEPSUM` \len => Pstring_FW len` can be rewritten as follows:

```
do len <- Buint32
  Pstring_FW len
```

3.3 Main Divergences from the Original DDC

3.3.1 Integration with Host Language. In the original DDC^α , the calculus and its host language are largely separate, with only specific places that host language expressions can appear (arguments of base types and function application, conditions in constrained types, and termination conditions of sequence types) and DDC^α 's syntax including its own separate definitions for variables, abstraction and application.

Notably, since DDC^α is a standalone DSL, host language expressions can not be used in place of types, disallowing the creation of types such as $\Sigma x : \text{Pbool} . \text{if } x \text{ then } \text{Pint} \text{ else } \text{Pstring}$ since if statements are not part of DDC^α itself. However, it is still possible to achieve the desired behaviour of this example through unions and constrained types: $\Sigma x : \text{Pbool} . \{ \text{Pint} \mid x \} + \{ \text{Pstring} \mid \text{not } x \}$. This highlights an important trade-off: while a smaller feature set is helpful in a calculus designed for doing proofs on, it can make actual implementations more verbose.

Since this DSL is embedded in Idris, it makes use of Idris' existing features where possible rather than re-implementing them. Specifically, abstraction and application are handled through Idris functions instead of being built into the data structure, parametrisation of base types is handled by making them functions from the parameters to unparametrised `DDCTypes`, and the binding of the value of τ_1 for τ_2 in dependent sums is handled by making τ_2 into a function from that value to an unparametrised `DDCType`. This simplifies the implementation of the DSL and gives users full access to Idris' features within it.

3.3.2 Consequences of Serialisation. While the original DDC^α did not include serialisation back into binary, this DSL aims to have support for this. As a result, some changes were needed.

First, some extra information is required by the DSL when defining types: base types requires a function to convert an instance back to binary (effectively the inverse of $\mathcal{B}_{\text{imp}}$), and some other types require functions to assign values to information lost during parsing - this discussed further in Section 3.7.

DDC^α originally defined dependent sums to have both the representation and parse descriptor values of τ_1 in scope when parsing τ_2 . Following this for the DSL would have made τ_2 a function from τ_1 's `Result` type to a `DDCType`. While the relevant case of `MkTy` has no issue with this, there is an issue when implementing the case of `RawTy`:

```
MkTy (DEPSUM t1 t2) = (x : MkTy t1 ** MkTy (t2 x))
RawTy (DEPSUM t1 t2) = (x : RawTy t1 ** RawTy (t2 ?missing))
```

The conversion from `MkTy` to `RawTy` is lossy, so there is no way to create the required input for τ_2 , other than perhaps constructing some arbitrary parsing metadata values. This can be fixed by instead making τ_2 depend on the `RawTy` representation of τ_1 : the `RawTy` case can be defined how the `MkTy` case originally was, and

MkTy can make use of `stripMetadata` to get the required input for τ_2 .

This does make the DSL slightly less powerful than the original DDC^α , as it is no longer possible to depend on the parse descriptor values of previous fields. However, that kind of behaviour is rarely seen in real-world file formats beyond simply checking if a field failed to parse, which can still be achieved through sum types.

3.3.3 Merging Representations and Parse Descriptors. One issue with using host-language abstraction, application and variables is that parametrised types become harder to reason about. For example, take the DDC^α type $\Sigma len : \text{Puint32} . \text{Pstring_FW}(len)$. Its representation type can be derived as follows:

$$\begin{aligned} & \llbracket \Sigma len : \text{Pint} . \text{Pstring_FW}(len) \rrbracket_{\text{rep}} \\ &= \llbracket \text{Pint} \rrbracket_{\text{rep}} * \llbracket \text{Pstring_FW}(len) \rrbracket_{\text{rep}} \\ &= (\mathcal{B}_{\text{type}}(\text{Pint}) + \text{none}) * (\mathcal{B}_{\text{type}}(\text{Pstring_FW}(len)) + \text{none}) \\ &= (\text{int} + \text{none}) * (\text{string} + \text{none}) \end{aligned}$$

Notably, $\mathcal{B}_{\text{type}}(\text{Pstring_FW}(len))$ is calculated during the final step of the evaluation while len is a free variable within the expression. DDC^α specifies that $\mathcal{B}_{\text{type}}$ can not depend on the parameter expression of the type, so the value of len is not needed to actually determine the result of this expression. Under the original DDC^α design, the calculus' types exist outside of the host-language, so it is possible to navigate through the structure of the type during translation and find `Pstring_FW` to determine the result should be string without needing to evaluate len . On the other hand, under the DSL design, `Pstring_FW( $len$ )` is a host language application of function `Pstring_FW` with parameter len , which leads to an issue: it is not possible to pattern match over function calls in Idris, and there is no value of len to evaluate this expression with.

Since Idris is a dependently typed language, this issue can be avoided through the use of a dependent pair. By defining the representation type as $(len : \text{Int} ** \text{RepTy} (\text{Pstring_FW } len))$, it can express the relationship required rather than assigning a concrete type to the right hand side in advance.

However, this solution does not work for parse descriptors, as they still depend on the value of the representation (stored in a separate data structure) rather than the value of another parse descriptor. To solve this, the DSL combines the representation and parse descriptor data structures into a single `Result` data structure, allowing for one dependent pair to handle both. This is discussed further in [Section 3.5](#).

3.3.4 Preserving Dependence in Host Language Types. DDC^α 's host language did not support dependent types, so the host-language representation of the types erased all dependency. For example, dependent sums are expressed simply as a regular tuple of their elements, and the base type `Pstring_FW( $n$ )` is simplified to just `String`, rather than a string of a specific length, since $\mathcal{B}_{\text{type}}$ is not allowed to change based on the parameter expression of the base type.

Since this DSL is targeting Idris, which has dependent types, all dependencies were preserved in the host language too. Specifically, dependent sum types are represented as dependent pairs in Idris, the representation type of base types is allowed to depend on their parameters, and constrained types are paired with a proof that their constraint was met.

3.3.5 No Recursive Types. As discussed in [Section 2.2](#), Idris will only evaluate type-level functions if they are proven to be total;

otherwise, they are just treated as constants. Introducing recursion to `DDCType` means it is no longer guaranteed that the data structure has an end point, and therefore functions over it such as `MkTy` and `RawTy` will not be total - this would be a major issue, since calls to them would no longer be evaluated down to their actual Idris types and therefore parsing outputs and serialisation inputs could not be given concrete types. This is similar to the issue discussed by Oury and Swierstra [15] in their Agha EDSL.

Therefore, recursive types were not implemented. This does reduce the DSL's expressivity, as file formats with features such as tree structures can no longer be described, though a large number of file formats are still possible to describe without recursion.

3.4 Binary Representation

Binary data is represented as a vector of booleans in the DSL - specifically, the type `Binary` is defined as $(\text{length} : \text{Nat} ** \text{Vect } \text{length } a)$. The use of individual bits simplifies accessing any subgroup of bits, and using the vector type helps to enforce that bounds checking is done where needed in parsing implementations.

This approach likely has large performance overheads compared to grouping the data in bytes or working with it through a C FFI interface like `PacketLang`, but it works well enough for a proof of concept.

3.5 The Result Type

The `Result` data type, which acts as a combination of the representation and parse descriptor types from DDC^α , is defined as follows:

```
record Result (tag : DDCTag) (tRep : Type) (tPdMisc : Type) where
  constructor MkResult
  rep : tRep
  err : Error
  sp : Span
  pdMisc : tPdMisc
  {auto @ prfRep : StayedOk tag (err.ec) rep}
```

The `rep` field stores the parsed value of the type. For composite types, the original DDC^α design places the representations of the child types within the parent's representation and the children's parse descriptors in the parent's parse descriptor. To avoid duplication, in the merged structure children's results are only included in the parent's representation and not in the parse descriptor.

Since the parse descriptors of all types included the `pd_hdr` field in DDC^α , the contents of this have been placed in the top level of the result type as the `err` (error code and count) and `sp` (span of offsets within the binary the value was parsed from) fields. The `pdMisc` field stores any remaining type-specific parse descriptor data - that is, anything from DDC^α 's $\llbracket _ \rrbracket_{\text{PD}}$ which is not `pd_hdr` or the parse descriptor of a child.

The `prfRep` field encodes a proof of the relationship between the representation field and the error code: if a result is marked as OK, then its representation must be in its successful form (for example, base types may not be none), and any child results must also be OK - this proof makes it sufficient to check the outermost error code to ensure that a parse was successful, and is also what enables the `stripMetadata` function to be created.

As discussed in [Section 2.2](#), the Idris compiler requires type-level functions to be total for them to actually be evaluated. Making use of `DDCType` directly within the definition of the `Result`

type created a mutual dependency that made the compiler unable to prove the DSL's core functions were total, so the type was designed in a way that could avoid this. Instead, the types of the `rep` and `pdMisc` fields are provided as parameters. Since the proof requirements of `StayedOk` do not depend on any of the actual parameters of the result's `DDCType`, just what constructor it uses, the simplified `DDCtag` type was created to indicate this without the same mutual dependence issues. For example, `getTag (DEPSUM t1 t2)` will return `T_DEPSUM`. The `MkTy` function is defined as follows to fill in these parameters:

```
MkTy t = Result (RepTy t) (PdMiscTy t) (getTag t)
```

`RepTy` and `PdMiscTy` are the analogues of DDC^α 's $\llbracket _ \rrbracket_{rep}$ and $\llbracket _ \rrbracket_{pd}$ functions respectively. The parametrisation allows for `RepTy`, `PdMiscTy` and `getTag` to be defined later without `DDCType` or `Result` directly depending on them, enabling the use of `DDCType` in them without issue.

The `StayedOk` type is given a constructor for each tag, consisting of a proof that any children are OK and that the representation is in its successful form, as well as a blanket failure constructor with the sole constraint that the given error code is not OK.

3.6 The Error Type

In DDC^α , error codes are described through a separate error code `ec` and error count `nerr`. To make it impossible for the number of errors to be greater than zero while the error code is still OK, the DSL instead chose to represent these through a combined Error type:

```
data Error : Type where
  MkOk : Error
  MkErr : (nerr : Nat) -> Error
  MkFail : (nerr : Nat) -> Error
```

The helper functions `ec : Error -> ErrorCode` and `nerr : Error -> Nat` are provided to allow the information to be read from an Error in the original style, where `ErrorCode` is an enum with values `OK`, `ERR`, `FAIL`.

To aid in proofs, `ErrorCode` implements the decidable equality interface. Additionally, the `decError` function is provided to perform this check over all error codes at once, returning the proofs as an instance of the following type:

```
data DecError : (ec : ErrorCode) -> Type where
  ItIsOk : ec = OK -> DecError ec
  ItIsErr : ec = ERR -> Not (ec = OK) -> DecError ec
  ItIsFail : ec = FAIL -> Not (ec = OK) -> DecError ec
```

3.7 Implementing the DSL's Types

3.7.1 Unit and Bottom. `UNIT`'s `RepTy` is Idris' own unit type, and its `RawTy` representation is the same, given that it can never fail.

`BOTTOM` is represented by `None`, a type with the sole value `MkNone` to indicate a failed parse. `BOTTOM`'s raw type is `Void`, as it should not be possible to represent a successfully parsed `BOTTOM`.

The `StayedOk` constructor for `UNIT` has no special requirements since `UNIT` can never fail to parse; no `StayedOk` constructor exists for `BOTTOM` since it is impossible for it to parse successfully.

Their parsing behaviour is unchanged from DDC^α , both ignoring the actual data and returning the sole value of their type, with fixed error codes of success and failure with 1 error respectively. Given the fixed value and error, Idris is able to automatically construct the `prfRep` field for each of these - `UnitWasOk` and `WasntOk` respectively.

Serialisation is similarly simple: `UNIT` returns an empty bit string, and `BOTTOM` is able to skip its implementation entirely by calling `void` with its value, given that its `RawTy` is `Void`.

3.7.2 Base Types. The interface for defining base types is slightly different to in the original DDC^α . Rather than defining functions like $\mathcal{B}_{type}$ and $\mathcal{B}_{imp}$ over all of the base types, the base types store these values within them directly. Additionally, parametrisation is done by creating Idris functions producing concrete `BASE_TYPE` instances rather than as part of the DSL itself. The unabbreviated `DDCType` constructor for them is:

```
BASE_TYPE : (bType : Type) -> {auto _ : Show bType} ->
  EncodeFn bType -> DecodeFn bType -> DDCType
```

It first takes the Idris type for a successfully parsed value `bType` (the analogue of $\mathcal{B}_{type}$), along with a proof that it implements the `Show` interface for debugging purposes (this is not strictly necessary and may be useful to remove for a practical application). It then takes a function to try parse a bit string into an instance of this type (the analogue of $\mathcal{B}_{imp}$), along with a function to encode an instance of this type back to a bit string (the serialisation analogue of $\mathcal{B}_{imp}$). These functions have the same type signatures as `parse` and `serialise` respectively.

`BASE_TYPE`'s representation type sums `bType` with `None` to represent when decode fails, like in DDC^α , though the order of the sum is reversed to follow the convention of `Either`'s left side being the error; the raw type consists of just `bType` since no error handling is needed. The `StayedOk` constructor requires that the representation was `Right` (i.e. was not `None`), allowing for `stripMetadata` to remove the `Either`.

The parse implementation simply forwards the binary and offset to decode, as DDC^α did with its $\mathcal{B}_{imp}$. Similarly, serialisation just forwards the value to encode.

3.7.3 Dependent Sums. As discussed in Section 3.3.4, the representation type of dependent sums preserves the dependence between the two component types. The dependence being on `RawTy` does introduce a slight complication here: since `stripMetadata` requires a parse to be successful, the right hand side can not exist without a successful parse of the left hand side. For example, `Pint `DEPSUM` \len => Pstring_FW len` would have no length to give the string if the first integer field failed to parse.

To handle this, the `MkTy` representation for `DEPSUM` uses a helper type named `Depsum`, a variant of dependent pair whose second item only exists if the error code of the first is `OK`:

```
data Depsum : (t1 : DDCType) -> (t2 : RawTy t1 -> DDCType)
-> Type where
  MkDepsumFull : (fst : MkTy t1) -> {auto 0 fstOk : fst.ec = OK}
-> (snd : MkTy (t2 $ stripMetadata {isOk=fstOk} t1 fst))
-> Depsum t1 t2
  MkDepsumPartial : (fst : MkTy t1) ->
{auto 0 fstNotOk : Not (fst.ec = OK)} -> Depsum t1 t2
```

It has two constructors, `MkDepsumFull` for when both items exist, and `MkDepsumPartial` for when only the first item exists;

both constructors take a proof about the first item's error code to ensure they are only used in the correct case. On the other hand, the `RawTy` representation does not need to handle the case where the first item failed, so it is able to use a plain dependent pair.

The `StayedOk` constructor for dependent sums requires a proof that the `MkDepsumFull` constructor is used (enabling `stripMetadata` to convert it to a plain dependent pair), as well as a proof that the second item is OK (the first item is implied to be OK by the fullness proof).

Parsing mostly follows the original DDC^α logic, though there are some slight differences. First, decidable equality is used in the error code checks to provide the proofs required by `Depsum` and `StayedOk`. Additionally, failing to parse the first item sets the overall error code to `FAIL` rather than just `ERR`, given that it now causes the second parse to be skipped entirely. Serialisation simply emits the serialisation of each item back-to-back.

3.7.4 Sums. Sums are represented identically to how they are in DDC^α , with both the `RepTy` and `RawTy` representations consisting of an `Either` combining their child types. `StayedOk` features two constructors for sums, each requiring a proof that the value is left or right respectively, and that said value is OK.

Parsing is mostly unchanged from DDC^α , other than again using decidable equality to aid in proofs. Serialisation just consists of emitting the serialisation for whichever item in the sum exists.

3.7.5 Intersections. Representation of intersections is again identical to DDC^α , using a tuple of the child types for both `RepTy` and `RawTy`. The `StayedOk` constructor just requires proofs that both children are OK.

Parsing again follows the DDC^α logic other than using decidable equality for proving `StayedOk`. Serialisation of intersections runs serialisation on both children and emits whichever output was longer. This is done so that types such as the intersection of an 8-bit and a 16-bit integer will not lose data by only emitting the shorter type.

3.7.6 Constrained Types. DDC^α represented constrained types as a sum with their child type on both sides, using left to represent constraint success and right to represent failure. In a dependently-typed host language, it can be useful to have the representation actually prove that the constraint was met, rather than just indicating it. For example, when placing a constraint over a value of type `Vect n a` that $n = 5$, a proof of the constraint can be used to refine the value's type to `Vect 5 a`.

`RawTy` makes use of Idris' `Subset` type to represent this, with the first item being the child type's value and the second item being a runtime-erased proof that the constraint was met. However, since the DSL's constraint test function is defined over the `RawTy` representation of the child type, `MkTy` runs into the same issue it did with `DEPSUM`: the type of the second element can not be defined if the first element fails to parse, since `stripMetadata` can not be called on it.

The `Constrained` type is used to handle this, with a similar approach to `Depsum`. It has two constructors: `MkConstrainedMet` - taking the first item, a proof it is OK (enabling `stripMetadata` to be used), and a proof the constraint was met - and `MkConstrainedUnmet` taking only a value for the first item. `Constrained` types' `StayedOk` constructor consists of a proof that the `MkConstrainedMet` constructor was used, which also implies that the underlying type was OK.

Parsing of constrained types is done by parsing the underlying type, then using decidable equality to first check it was

successful, then to provably check whether the constraint was met. Serialisation just consists of emitting the serialisation of the underlying type.

3.7.7 Sequences. As in DDC^α , sequences are represented by a pair containing a length and a list of that length, though in the DSL this is a dependent pair so that the list can be typed as a `Vect` of said length. This representation is used for both `MkTy` and `RawTy`.

Defining sequence types requires some additional information to DDC^α , with their full `DDCType` constructor being as follows:

```
SEQUENCE : (tElem : DDCType) -> (tSep : DDCType) ->
  (mkSep : SeqRaw tElem -> RawTy tSep) ->
  (termCon : SeqRaw tElem -> Bool) ->
  (tTerm : DDCType) ->
  (mkTerm : case getTag tTerm of
    T_BOTTOM => ()
    _ => Binary -> SeqRaw tElem -> RawTy tTerm
  ) -> DDCType
```

`SeqRaw` is a shorthand for the `RawTy` representation of a sequence, $(n : \text{Nat} ** \text{Vect } n (\text{RawTy } t\text{Elem}))$. As in DDC^α , the DSL's constructor takes the element, separator and terminator types and a termination condition. On top of this, it takes functions `mkSep` and `mkTerm` to allow customisation of separator and terminator values when serialising.

`mkSep` is provided the list of elements emitted so far to produce a value of the separator type to serialise. Similarly, `mkTerm` is provided with the complete sequence to produce a value of the terminator type. When `tTerm` is `BOTTOM` (i.e. sequences that either run until their termination condition or until the end of the file), `mkTerm` is not required, since it would have to create an instance of `Void`.

The `StayedOk` constructor checks that all elements of the sequence were OK, done using Idris' `All` type to apply a proof across the elements of the vector.

Outside of the usual tweaks to support proofs, the only difference in the parsing implementation from DDC^α is that sequences do not continue to be parsed after one element fails. This is because `termCon` is parametrised by the `RawTy` representation, and therefore could not be called on a sequence where some elements have failed; immediate termination seemed more useful than potentially parsing the entire remaining file contents as garbage elements of the sequence when `termCon` can not be checked.

Unfortunately, the implementation of parsing for sequences, and therefore the parse as a whole, is unable to be proven as total. This is because the current typing of `parse` does not prove that the new offset returned will not be less than the starting offset. Sequence parsing is structured around the recursive function `continue`, which runs once per element to check if the sequence is complete parses another element if not. There is no guarantee that a sequence will always reach a terminator type or satisfy their termination condition, so proving `continue` is total would require proving that the end of the file would be reached if neither of these are met first. However, due to the aforementioned weakness of `parse`'s typing, there is no guarantee that parsing an element will actually move toward the end of the file, so this is not possible to prove. Additionally, adding this restriction onto `parse` may limit the expressivity of the DSL.

Serialising sequences consists of emitting the individual elements' bit strings, interspersed with the outputs of `mkSep`. At the end of the sequence, `termCon` is checked. If it returns true,

the sequence is left as-is; if it returns false, `mkTerm` is used to generate a terminator to append.

3.7.8 Computed Types. Computed types are fairly trivial, as they are just computed from Idris expressions. Their `RawTy` and `MkTy` representation types are the type of the expression provided, parsing just consists of evaluating said expression, and their serialised form is an empty bit string. Their `StayedOk` constructor places no requirements beyond them using the `COMPUTE` `DDCType` constructor.

3.7.9 Absorbed types. Under the original DDC^α design, absorbed types would be represented `Either None ()`, where unit represents the child type parsing successfully and none represents it failing. With this typing, `StayedOk` would not be able to guarantee that parse actually succeeded in parsing the underlying type instead of just returning `()` with an arbitrary successful parse descriptor. Instead, the DSL replaces the unit type with a `Subset` of unit and an erased copy of the child's value. This avoids needing to keep the child's value around in memory at runtime, but still gives `StayedOk` access to checking its error code was OK (since it's an erased proof itself). Now, the guarantees of `StayedOk` can be propagated through absorbed types with no runtime overhead. The `RawTy` representation does not need to represent the failure case and does not need to prove itself to `StayedOk`, so it can use the plain unit type.

Parsing follows DDC^α 's logic, again with decidable equality added. However, since parsing of absorbed typed is inherently lossy, serialisation requires additional information to be added when defining the types. The `DDCType` constructor `ABSORB` contains the new parameter `mkVal : RawTy t` to provide a value of the child type, and serialisation consists of emitting the bit string generated from this value.

3.7.10 Scan Types. The `MkTy` representation and parsing logic for scan types are unchanged from DDC^α besides the usual reversal of `Either` and addition of decidable equality; the `RawTy` representation is just the child type since the failure case is not needed. The `StayedOk` constructor just requires that the value is not none, and that the child error code is OK.

Serialising scan types simply places the child type's bit string immediately at the current offset; DDC^α defines parsing a scan with a non-zero offset as an error, making it impossible to represent one in `RawTy`, so there is no need to handle those in serialisation.

4 Evaluation

4.1 Implementing Base Types

To illustrate the system for DSL users to define base types, a function to generate a `BASE_TYPE` instance for any Idris type implementing the `FiniteBits` interface was created. For simplicity, only big-endian values are considered here, though a similar approach would work for little-endian too.

4.1.1 The FiniteBits Interface. `FiniteBits` allows for types representable as a fixed number of bits to expose their binary representation. Notably, it defines the `bitSize` function to get the length of a type in bits, and the `asBitVector` and `fromBitVector` functions to convert instances of the type to and from vectors of booleans representing their bits, neither of which may fail.

These vectors are ordered from least significant bit to most significant bit across the vector as a whole, so the custom `asBinary`

and `fromBinary` functions are defined to compose them with `reverse`, giving standard big-endian representations.

4.1.2 The Generator Function. Generation is done using the `finiteBitsBaseType` function, with the following top-level definition:

```
finiteBitsBaseType : (a : Type) -> {auto _ : Show a} ->
{auto _ : FiniteBits a} -> DDCType
finiteBitsBaseType a = BASE_TYPE a encode decode where ...
```

It takes the Idris type `a` to generate a `DDCType` for, along with proof that type implements the `Show` interface (required by `BASE_TYPE`) and the `FiniteBits` interface. It constructs a `BASE_TYPE` with representation type `a` and uses helper functions `encode` and `decode` to define parsing and serialisation.

4.1.3 Encoding. The implementation of encoding is straightforward, as no error handling is needed. It uses the aforementioned `asBinary` function to convert the value to bits, and pairs it with its length to match the DSL's Binary format:

```
encode : a -> Binary
encode x = vectAndLength $ asBinary x
```

4.1.4 Decoding. Decoding is implemented as follows:

```
decode : Binary -> Offset -> (Offset, BaseResult a)
decode (len ** b) w = case w + bitSize `isLTE` len of
  Yes prf => let val = fromBinary $ slice w bitSize b {prf}
             w' = w + bitSize {a}
             in (w', MkResult (Right val) MkOk (MkSpan w w') ())
  No _ => (w, MkResult (Left MkNone) (MkFail 1) (MkSpan w w) ())
```

It starts by checking that enough bits exist in the binary after the given offset to parse an instance of this type. Operations on vectors require proofs that the indices used are in bounds, so the check is implemented using decision function `isLTE` to prove this. When not enough bits are available, a failing result is returned with the offset unchanged. When they are available, the bits are passed to the `fromBinary` function to get its parsed value and the offset is incremented by the size of the type. A successful result is then returned, with `StayedOk` automatically constructed by Idris since the `rep` field of the result is guaranteed to be `Right`. The bits are extracted using helper function `slice`, which is a wrapper for Idris' take and drop functions with derivation of their individual required proofs.

4.1.5 Implementing Concrete Base Types. This generator function can then be used to trivially build a concrete `BASE_TYPE` instance for any Idris type implementing the interface. Idris provides fixed-width integer types `Bits8-Bits64` (unsigned) and `Int8-Int64` (signed) along with implementations of `FiniteBits` for them, so these can be passed directly to the generator:

```
Buint64 : DDCType
Buint64 = finiteBitsBaseType Bits64
...
Bint8 : DDCType
Bint8 = finiteBitsBaseType Int8
```

The DDC^α paper had the convention of naming its base types with a `P` prefix, and all of its base types were based on first parsing

the bits to ASCII. In a similar style, the project's base types which are parsed from binary directly are prefixed with a B; the P prefix is still used for any ASCII-based types.

4.1.6 Summary. This example highlights the flexible interface for programmers to define their own base types. The safety guarantees are extended into users' implementations by requiring the results they output have valid StayedOk proofs like any other, and also by typing the binary as a vector to prevent out-of-bounds reads. It also highlights the power of giving full access to Idris' features in the DSL, with functions and interfaces enabling simple creation of base types for a variety of host-language types.

4.2 Describing PNG Files

To show how file formats can be described using the DSL, a description of PNG files is given.

4.2.1 PNG Specification. The PNG file format[16] consists of a constant 8-byte file format signature (hexadecimal 89 50 4e 47 0d 0a 1a 0a), then a sequence of chunks for the rest of the file. These chunks consist of a length (represented as a 4-byte integer), a type (4 1-byte ASCII letters), a body of the specified length in bytes, and a checksum of the type and body (another 4-byte integer). The structure of the body depends on the type of the chunk. With the exception of checksums, PNG requires that all four-byte unsigned integers are limited to the range 0 to $2^{31} - 1$ (known as a PNG four-byte unsigned integer).

4.2.2 File Signature. The first step of implementing a DSL description for a PNG is representing the signature. Parsing the data itself can be done using a sequence of Buint8, and a constraint can be added on top of that to check the value is correct. However, this runs into a weakness in the sequence type: vector equality in Idris is only defined for vectors of the same length, and the representation of sequences does not carry a proof that their termination condition was met, so it can not be proven that the vector parsed is actually of length 8. In fact, this proof could not be added under the current semantics, since sequences may terminate without error by reaching the end of the binary instead of meeting their termination condition. Instead, the sequence can be wrapped in an additional constraint type to prove the length requirement was actually met - the ARRAY function was created as a shorthand for this. The signature can then be described as follows:

```
ARRAY Buint8 8 # \((8 ** vals) `Element` _) =>
  vals == [0x89, 0x50, 0x4e, 0x47, 0x0d, 0x0a, 0x1a, 0x0a]
```

4.2.3 Chunks. Next, a description of chunks is needed.

PNG 4-byte integers can be defined using a simple constraint over 32-bit unsigned integers: Buint32 # (<= 0x7fffffff).

Chunk tags are defined to only consist of ASCII letters. This can be enforced using the following proof type, where the only possible constructors are for lowercase and uppercase letters:

```
data IsChunkChar : Char -> Type where
  IsLowercase : {auto prf : isLower c = True} -> IsChunkChar c
  IsUppercase : {auto prf : isUpper c = True} -> IsChunkChar c
```

Decision function isChunkChar is then defined to provably check this constraint for a given character.

Since chunk tags determine the type of the chunk body, it can be useful to pattern match over their values. Pattern matching over vectors is inconvenient as it requires explicitly writing them out each time, so an enum representation of the tags can help make this more concise:

```
data ChunkType = CT_IHDR | CT_PLTE | CT_IDAT | CT_IEND | ... |
  CT_UNKNOWN (Subset (Vect 4 Char) (All IsChunkChar))
```

The CT_UNKNOWN constructor is provided to enable handling unknown chunk types (such as 3rd-party extensions) as vectors of letters still. Helper functions fromChunkName and toChunkName were then defined to transform between character vectors and this representation. A custom base type can then be defined to parse and serialise chunk types in this representation:

```
BchunkType : DDCType
BchunkType = (BASE_TYPE ChunkType encode decode) where
  raw : DDCType
  raw = (ARRAY Pchar 4)
  encode : ChunkType -> Binary
  encode ty = serialise raw $ MkArray (toChunkName ty).fst
  decode : Binary -> Offset -> (Offset, BaseResult ChunkType)
  decode b w = let
    -- Parse underlying chars
    (w', res) = assert_total $ parse raw b w
    Yes isOk = res.ec `decEq` OK
    | No _ => (w', BaseFail (MkFail 1) res.sp)
  (4 ** name) `Element` _ = stripMetadata raw res
  -- Check chars are in bounds
  Yes charsInRange = all isChunkChar name
  | No _ => (w', BaseFail (MkFail 1) res.sp)
  -- Convert to enum
  rep = fromChunkName (name `Element` charsInRange)
  in (w', BaseOk rep res.sp)
```

parse not being total does pose an issue here: the base type becoming non-total would make RawTy and MkTy unable to resolve its type, making descriptions involving it harder to use. To avoid this, the parse call had to be asserted as total. This should be safe, since parsing an array of 4 characters should always terminate, but it is not an ideal solution.

Additionally, a large amount of boilerplate code was required here just for mapping a type to an enum. This suggests adding a dedicated DSL feature similar to PacketLang's Decodable system to abstract this could be beneficial.

Using dependent types, the body's type can be defined as a function of the chunk type value. For example, implementing a specialised DDCType ihdrChunk for IHDR chunk bodies:

```
chunkBodyTy : ChunkType -> Nat -> DDCType
chunkBodyTy CT_IHDR length = ihdrChunk
chunkBodyTy _ length = ARRAY Buint8 length -- Unimplemented types
```

Ideally, a constraint should be placed over the checksum field of the chunk to check it is correct. This requires access to the underlying bytes of the relevant fields, which can be achieved by typing them as intersections with arrays of bytes. For the body field, this has the added benefit of ensuring the whole body is parsed even if chunkBodyTy returns a DDCType that parses less bytes than required.

The overall DDCType for chunks can then be defined as follows:

```
chunk : DDCType
chunk = with DDCType do
```

```
length <- pngUInt32
chunkType <- BchunkType .& ARRAY Buint8 4
body <- chunkBodyTy (fst chunkType)
      .& ARRAY Buint8 (cast length.fst)
Buint32 #
  (== calcCrc ((snd chunkType).fst.snd ++ (snd body).fst.snd))
```

This description does reveal some weaknesses in the DSL. First, creating an instance of a chunk to serialise is not ideal, as it requires manually producing the byte arrays alongside the actual values for the fields covered by the checksum; it may be beneficial to allow serialisation to only provide one element in intersections. Second, nothing prevents the DDType returned by chunkBodyTy from parsing more bytes than it is supposed to, as intersections simply take the end offset of whichever child parsed the furthest.

4.2.4 Top-Level Description. Using these definitions, the overall DDType for a PNG file can be defined as follows:

```
png : DDType
png = with DDType do
  -- Signature
  ARRAY Buint8 8 # \((8 ** vals) `Element` _) =>
    vals == [0x89, 0x50, 0x4e, 0x47, 0x0d, 0x0a, 0x1a, 0x0a]
  -- Chunks, only terminate at end of file
  SEQUENCE chunk UNIT mkUnitSep (const False) BOTTOM ()
  # validChunkSet
```

validChunkSet performs checks on the number, quantity and mutual exclusion requirements of chunks defined in the PNG specification. The generic typing of chunks allows for this function to iterate over the chunks and pattern match on their type field to identify them. Some of the restrictions depend on values in the IHDR chunk, which the function is able to locate through this pattern matching, and the chunkBodyTy dependenc typing allows it to access the body represented as an ihdrChunk instance to cleanly retrieve these fields once the chunk has been proven to be an IHDR.

4.2.5 Testing. This description was able to successfully parse a small PNG file, including the specialised parsing of its IHDR field, and serialise the result back to an identical binary. Additionally, throughout development, the parse descriptor system was able to accurately identify why parses failed, aiding in the discovery of bugs in the PNG type.

One issue encountered was that the compile times for the PNG source file in partiular became extremely slow (in the order of minutes). It is not currently clear what the bottleneck here is.

4.3 Embedding PacketLang

To demonstrate the expressivity of the DSL, an embedding of Fowler [8]’s PacketLang was created in it. The approach taken was to create a mapping from the PacketLang data structure to the project’s constructs, and adjust PacketLang’s mkTy, unmarshal and marshal functions to use this mapping to forward their inputs to RawTy, parse and serialise respectively.

PacketLang was originally written for Idris 1, so some parts needed adjusting due to language changes rather than the embedding. Most notably, Idris 2 removed syntax macros, so some of PacketLang’s syntax had to be simplified.

4.3.1 PacketLang Data Structure. Translation of the PacketLang data structure was handled by the toDDC function.

Its first constructor is CHUNK, which represents an atomic piece of data such as a string. The exact data format is described through its own Chunk data structure, so translation defers this to a separate chunkToDDC function, discussed further in Section 4.3.2.

IF chooses between two child PacketLang instances depending on whether a provided Idris expression provided is true or false. This was able to be translated directly to an Idris if-statement, with toDDC being recursively called on both branches’ child types.

The LIST constructor will repeatedly parse a PacketLang instance until an element fails to parse, at which point it sets its offset back to the starting point of that element and considers the list parse successful. This behaviour does not directly map the DSL’s SEQUENCE as either a termination condition or terminator type, but a special terminator type can be designed to recreate this lookahead behaviour, using the same approach Fisher et al. [7] took to embedding DATASCRIP arrays in DDC^α. The type not t = (t .+ UNIT) # isRight will only be considered successful if t fails to parse (since the left side of sums is tried first, therefore isRight would be false if t parsed). The terminator type of the sequence can then be set to the not of the element type to recreate the desired lookahead behaviour; the termination condition is set to always be false.

The remaining constructors (//), LISTN, NULL and »= all map directly to the DSL’s SUM, ARRAY, UNIT and DEPSUM respectively, other than calling toDDC on their child types.

4.3.2 Chunk Data Structure. Translation of the Chunk data structure was handled by the chunkToDDC function.

The Bit constructor represents an n-bit integer, and was translated as a custom base type Bbit, with a decode function based on its original unmarshal implementation. Similarly, boolean CBool was given a custom base type Bcbool, which simply tests the bit at the given offset.

CString and LString map directly to Pstring "\0" and Pstring_FW respectively.

The PROP constructor allows for testing constraints and getting proof that they passed. While PacketLang is able to construct a range of proof types with its propositions, the DSL’s constraints are limited to proving that a boolean function returned true. To handle this, PROP was encoded as a COMPUTE of the original unmarshalProp function responsible for construction the proofs, wrapped in a constraint that the unmarshal was successful. This does not lose functionality over the original, but it suggests there is room for a more powerful proof testing system in the DSL.

Decodable is a specialised version of Bit where the resulting integer is mapped to and from to another type (typically an enum) through user-provided functions. This was handled through another custom base type, Bdecodable, which wraps Bbit with applications of these user-provided translation functions. Its design is similar to BchunkType from Section 4.2.3, including the need to assert totality in the same way.

4.3.3 Testing. To validate the embedding, the tests from Appendix A.3 of the original paper[8] were ported (adjusting for Idris 2 changes and slight type differences in the DSL) and ran, all of which passed. Additionally, the DNSParser code was ported and the example DNS packet from Appdenix A.4 was able to be parsed with identical output to the original paper, and then re-marshalled to a binary identical to the input.

4.4 Correctness

4.4.1 Parsing Correctness. Fisher et al. [7] defined two core correctness properties for DDC^α . First, the outputs of the parsing function for type τ should have the types specified by $\llbracket \tau \rrbracket_{\text{rep}}$ and $\llbracket \tau \rrbracket_{\text{PD}}$ - the typing of the parse function proves that this must be true for the DSL. Additionally, parser outputs should be in canonical form, implying that parse descriptors reflect any errors in the representation. The DSL does not quite follow this property, as it does not place strict requirements on error counts outside of them being 0 when the error code is OK, but it does prove a similar property through `StayedOk`: if a result has error code OK, its representation will be in its successful form, and all of its children's error codes will be OK. A corollary of DDC^α 's second property is that checking the outermost error code is zero is sufficient to prove there are no errors in a canonical-form parse result; similarly, the DSL proved that checking that the outermost error code is OK is sufficient to prove all components of the result are OK (though checking that the outermost error count is 0 is not sufficient, since ERR and FAIL do not guarantee non-zero error codes).

4.4.2 Kinding. DDC^α also defined a kinding system to ensure correct usage of its types. In the DSL, these rules are expressed through the Idris types given to the `DDCType` constructors, and therefore automatically enforced by the compiler.

4.4.3 Serialisation Correctness. A desirable property for serialisation is a round-trip property - that is, proof serialisation is an inverse operation of parsing. Due to features such as absorbed types, the DSL's parsing is inherently lossy, so a Parse-then-Serialise round trip property is not possible, but a Serialise-then-Parse property is still desirable.

All Serialise-then-Parse tests done during the evaluation were able to pass, so the semantics do work to some extent. However, the current design has some flaws that prevent this being proven to always hold. For example, `Right -1` is a valid `RawTy` instance of the `Buint8 .+ Bint8` type, but the binary it produces would be able parsed `Left 0xff` since sum parsing is defined to try the left type first and this binary is also a valid representation of the left type. Similarly, `(2 ** [1, 1])` is a valid instance of a `Buint8` sequence with a termination condition of length being greater than 0, but would be re-parsed as `(1 ** [1])` (i.e. sequence instances do not carry a proof that their termination condition would not become true early). Additionally, `(1, 2)` is a valid instance of `Buint8 .& Buint8`, but would be re-parsed as `(1, 1)` since only one item's serialisation is emitted.

5 Conclusions

5.1 Summary and Reflections

A DSL was created based on DDC^α , embedded in Idris 2 and covering all features other than recursive types. A dependently typed host language enabled precise output types to be computed for parsing, and precise input types to be computed for serialisation. Additionally, it allowed the host-language representation types to encode additional information compared to DDC^α , such as preserving the relationships in dependent sums and providing proofs that constraints were met.

The kinding and parser type-correctness rules are able to be automatically checked by Idris' type system, and a slightly weaker variant of DDC^α 's canonical form proof is proven-by-construction with the DSL's `StayedOk` type. This enabled the creation of the `stripMetadata` function, which allows for access

to the plain representation of data when a parse has been checked to be successful, and also enabled said check to consist of merely inspecting the outermost error code.

Totality of the parsing function was not able to be proven, which is unfortunate as it would have ruled out the possibility of denial-of-service attacks on poorly designed types.

Serialisation semantics were also introduced, and they are able to successfully re-encode some data structures, though there are known flaws.

The DSL was successfully able to describe, parse and serialise the PNG format, and host an embedding of `PacketLang` that correctly parsed and serialised DNS packets with minimal adjustments to the original code.

5.2 Future Work

A key improvement that could be explored is introducing additional proofs and restrictions to guarantee the totality of parsing and the Serialise-then-Parse round-trip property.

Exploration of recursive types could also be quite beneficial, as many more file formats become possible to describe in the DSL if they can somehow be implemented.

Additionally, making use of Idris' FFI system to handle bit manipulation in C like `PacketLang` [8] could be beneficial to runtime performance.

Compile times can also grow quite long when using the DSL, as seen with the PNG evaluation; discovering and fixing bottlenecks in this area could greatly improve the developer experience, since Idris is designed for type-driven development supported by the compiler[2].

It is also likely that some of the types and proofs in the codebase could be simplified, particularly those written early on. This would primarily be for readability, though it may also be beneficial to compile times.

The evaluation also highlighted various areas with room for improvement of user experience in the DSL, such as the awkwardness of serialising unions and working with fixed-length sequences. For example, it may be beneficial to create more specific variants of the sequence type where a termination condition being met is mandatory for a successful parse, enabling it to carry a proof the condition was met. Similarly, functionality like `PacketLang`'s `Decodable` to enable refinement of other parsing types, such as mapping integers to an enum, could be convenient. Allowing constraints to have more specialised proof types like in `PacketLang`'s proposition system, such as proving a field is less than another through the `LT` type instead of the less powerful `(a < b) = True` type, would also be beneficial.

A notable feature in similar work with no equivalent in the DSL is McCann and Chandra [13]'s overlay system, enabling the layering of additional semantics onto existing types. For example, this would have been useful to allow the PNG description to be extended without new chunk types without having to modify the original PNG type definitions.

Acknowledgments

Thanks to Simon Fowler for his support and feedback throughout this project.

References

- [1] Emil Axelsson, Koen Claessen, Gergely Dévai, Zoltán Horváthy, Karin Keijzer, Bo Lyckegårdz, Anders Persson, Mary Sheeran, Josef Svenningsson, and András Vajdax. 2010. Feldspar: A domain specific language for digital signal processing algorithms. In *Proceedings of the Eighth ACM/IEEE International*

- Conference on Formal Methods and Models for Codesign (MEMOCODE '10)*. IEEE Computer Society, USA, 169–178. doi:10.1109/MEMCOD.2010.5558637
- [2] Edwin Brady. 2017. *Type Driven Programming with Idris*. Manning Publications. <https://www.manning.com/books/type-driven-development-with-idris>
 - [3] Edwin Brady. 2021. Idris 2: Quantitative Type Theory in Practice. arXiv:2104.00480 [cs.PL] <https://doi.org/10.48550/arXiv.2104.00480>
 - [4] Edwin C. Brady. 2011. IDRIS — systems programming meets full dependent types. In *Proceedings of the 5th ACM Workshop on Programming Languages Meets Program Verification* (Austin, Texas, USA) (PLPV '11). Association for Computing Machinery, New York, NY, USA, 43–54. doi:10.1145/1929529.1929536
 - [5] Peter Dybjer and Anton Setzer. 1999. A Finite Axiomatization of Inductive-Recursive Definitions. In *Typed Lambda Calculi and Applications*, Jean-Yves Girard (Ed.). Springer Berlin Heidelberg, Berlin, Heidelberg, 129–146. https://doi.org/10.1007/3-540-48959-2_11
 - [6] Kathleen Fisher and Robert Gruber. 2005. PADS: a domain-specific language for processing ad hoc data. *SIGPLAN Not.* 40, 6 (June 2005), 295–304. doi:10.1145/1064978.1065046
 - [7] Kathleen Fisher, Yitzhak Mandelbaum, and David Walker. 2010. The next 700 data description languages. *J. ACM* 57, 2, Article 10 (Feb. 2010), 51 pages. doi:10.1145/1667053.1667059
 - [8] Simon Fowler. 2014. *Verified Networking using Dependent Types CS4099: Major Software Project*. BSc Dissertation. University of St Andrews. <https://simonjf.com/writing/bsc-dissertation.pdf>
 - [9] P. Hudak. 1998. Modular domain specific languages and tools, In *Proceedings. Fifth International Conference on Software Reuse* (Cat. No.98TB100203). *International Conference on Software Reuse*, 134–142. doi:10.1109/ICSR.1998.685738
 - [10] Stefan Höck and Idris Community. 2021. Refined Primitives - Idris 2 Tutorial. <https://idris-community.github.io/idris2-tutorial/Tutorial/Prim/Refined.html> Accessed: 2026-05-04.
 - [11] D. Curtis Jamison. 2003. Structured Query Language (SQL) Fundamentals. *Current Protocols in Bioinformatics* (2003), 9.2.1–9.2.29. doi:10.1002/0471250953.bi0902s00
 - [12] Yitzhak Mandelbaum, Kathleen Fisher, David Walker, Mary Fernandez, and Artem Gleyzer. 2007. PADS/ML. *ACM SIGPLAN Notices* 42 (1 2007), 77–83. Issue 1. doi:10.1145/1190215.1190231
 - [13] Peter J. McCann and Satish Chandra. 2000. Packet types. *ACM SIGCOMM Computer Communication Review* 30 (8 2000), 321–333. Issue 4. doi:10.1145/347057.347563
 - [14] Marjan Mernik, Jan Heering, and Anthony M. Sloane. 2005. When and how to develop domain-specific languages. *ACM Comput. Surv.* 37, 4 (Dec. 2005), 316–344. doi:10.1145/1118890.1118892
 - [15] Nicolas Oury and Wouter Swierstra. 2008. The power of Pi. In *Proceedings of the 13th ACM SIGPLAN International Conference on Functional Programming* (Victoria, BC, Canada) (ICFP '08). Association for Computing Machinery, New York, NY, USA, 39–50. doi:10.1145/1411204.1411213
 - [16] Portable Network Graphics (PNG) Working Group. 2025. *Portable Network Graphics (PNG) Specification (Third Edition)*. Technical Report. W3C. <https://www.w3.org/TR/png-3/>
 - [17] Kaitai Team. 2015. Kaitai Struct: declarative binary format parsing language. <https://kaitai.io/> Accessed: 2026-05-04.
 - [18] Marcell van Geest and Wouter Swierstra. 2017. Generic packet descriptions: verified parsing and pretty printing of low-level data. In *Proceedings of the 2nd ACM SIGPLAN International Workshop on Type-Driven Development* (Oxford, UK) (TyDe 2017). Association for Computing Machinery, New York, NY, USA, 30–40. doi:10.1145/3122975.3122979